AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

Values variables and types

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Inspect the type. Trace the binding. Explain the task meaning.

Distinguish numbers, text and booleans without treating earlier assignments as live formulas.

Keep one habit: Type table → count/tag operations → saved-value trace → five cases → selected independent script.

MAIN LESSON · SECTION 1

What value does this name hold right now?

> Types guide operations, while the task determines whether the value makes sense.

Your learning goals

I can distinguish integer, floating-point, text and Boolean **value**s.

I can trace **reassignment** without treating a name as a live formula.

I can choose a suitable representation for a count and an identifier.

Recall before notes

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Without opening the old lesson, explain one key idea from From a paper algorithm to Python. Give an example and a mistake that the idea helps you avoid.
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- This week you will practise:
- Distinguish integer, floating-point, text and Boolean **value**s
- Trace **reassignment** without treating a name as a live formula
- Choose a suitable representation for a count and an identifier
- What does print(total) do after total = 24?

> **My starting explanation**

Your end goal

Trace **value**s **variable**s and **type**s and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

A count and an identifier can look similar

> Quotes preserve the text identity of a shelf label such as 08.

The count-versus-label task

A count and a shelf label can look similar while needing different operations. The integer 8 is a number; "08" is text that preserves a leading zero.

You can count eight objects, but adding two shelf labels is usually meaningless. This week you will examine **value**s, trace changes to names and explain which representation a task needs.

Use the saved script and inspect the value types

Use the saved Python 3 script workflow from Week 1. Make a new file **value**s.py, predict before running and keep actual **output** separately. Use fictional labels only.

Four useful **type**s are int for whole numbers, float for floating-point numbers, str for text, and bool for True or False. A **type** describes a **value**, not whether its meaning is sensible for the task.

Your explanation

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Read the literal and its type before choosing an operation

> The displayed characters alone do not identify the value’s type.

Compare four built-in types

Literal in code	Type	Possible meaning
8	int	Number of counters
2.5	float	Length in metres
"08"	str	Shelf identifier
True	bool	A yes-or-no condition

Complete type-inspection program

```
count = 8
shelf = "08"
print(type(count))
print(type(shelf))
```

Read quotes, booleans and printed type labels

type(count) produces the **type** of the **value** bound to count. These print lines display <class 'int'> and <class 'str'>. The quotes in "08" mark text; they are not displayed by print(shelf).

True and False have capital initials and no quotes. "False" is text, not a Boolean.

Explain the deciding check

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare saved values with later reassignment

> An earlier computed value is not a live formula that follows another name.

Predict the supported change

- Trace the five-line count and saved program. Record both printed **output**s, then predict the result of assigning saved = count after the change. Check with a run or label a paper trace.

Keep the procedure fixed. Explain what changed in the inputs and result.

Predict the pictured variant

- Change: Change count to 10, leaving tag unchanged.

▼ **Check after predicting**

Prediction: The two valid **output**s become 12 and 82.

Reason: Only the integer **variable** changes; the **string** still contains the character 8.

▼ **Earlier wording, alternatives and picture transcripts**

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace numbers and text through the same-looking plus sign

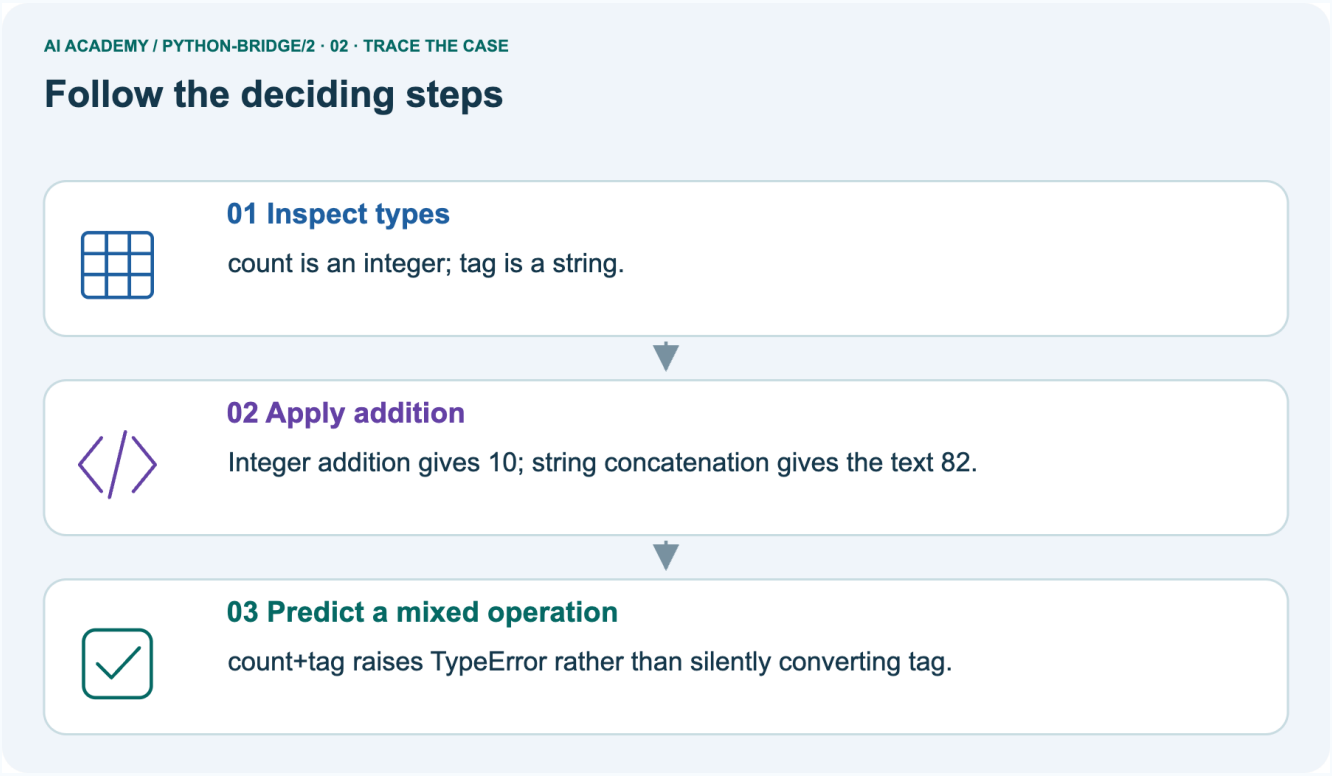
> The type changes what the operation means.

Read the complete count/tag record

A number and its printed-looking text. A program stores a numeric badge count and a **string** that looks like that count.

Python 3: count=8; tag="8"; print(count+2); print(tag+"2").

Trace the picture and type rules



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Inspect types	count is an integer; tag is a string.
2. Apply addition	Integer addition gives 10; string concatenation gives the text 82.
3. Predict a mixed operation	count+tag raises TypeError rather than silently converting tag.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Values carry **type**s, not merely appearances. The plus operator performs numeric addition for these integers and concatenation for these **string**s; the mixed operation is invalid here.

Read the complete runnable comparison

```
count = 8
tag = "8"
print(count + 2)
print(tag + "2")
```

Inspect valid results and the mixed-type failure

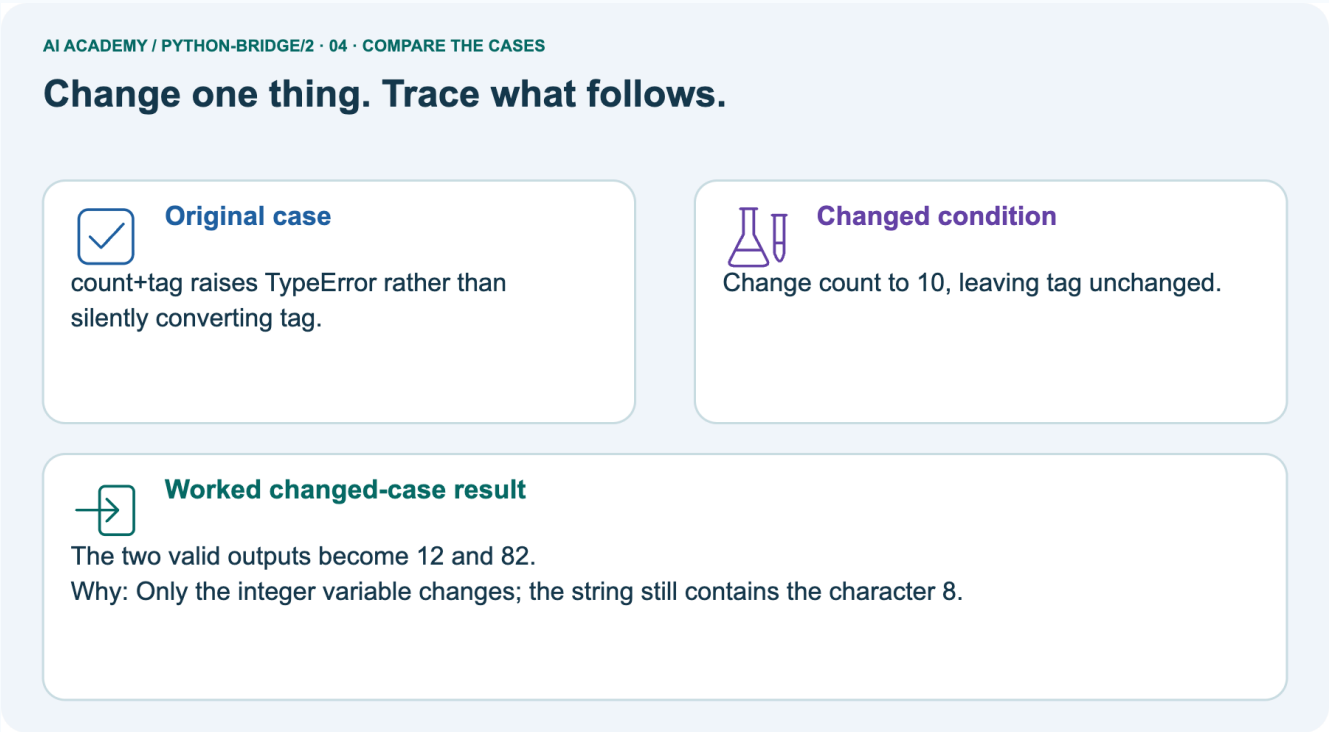
Expression	Operation under these supplied types	Result
count + 2	Integer addition	10
tag + "2"	String concatenation	Text 82

Expression	Operation under these supplied types	Result
count + tag	Unsupported int+str addition	TypeError, no silent conversion
After count=10, tag unchanged	Only the integer input changes	12 and text 82; tag still "8"

Predict changing only count

- Change one thing: Change count to 10, leaving tag unchanged. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

State built-in-type scope and explain

- These examples use built-in integers and **string**s; operator behaviour depends on **type**s and is not proof of general model reasoning.

- Read the labelled picture: Predict both print **output**s and explain why they differ. Does count+tag work, and what happens to each valid **output** after count becomes ten?

▼ **Separate saved-value trace**

A saved value is not a live link

INSTRUCTION	COUNT NOW	SAVED NOW
count = 8	8	Not assigned
saved = count	8	8
count = count + 2	10	8

Evaluate the right side using the current value, then bind the left-hand name. The saved integer does not update when count is reassigned.

```
count = 8
saved = count
count = count + 2
print(count)
print(saved)
```

The **output**s are 10 and 8 on separate lines. saved = count uses the **value** available at that moment.

It does not store a formula that keeps following count. count = count + 2 is valid assignment: read the old 8, calculate 10, then rebind count. It is not a claim that 8 equals 10.

> **After count becomes 10, add saved = count and print(saved). Predict the new output and explain why this is different from expecting saved to change automatically.**

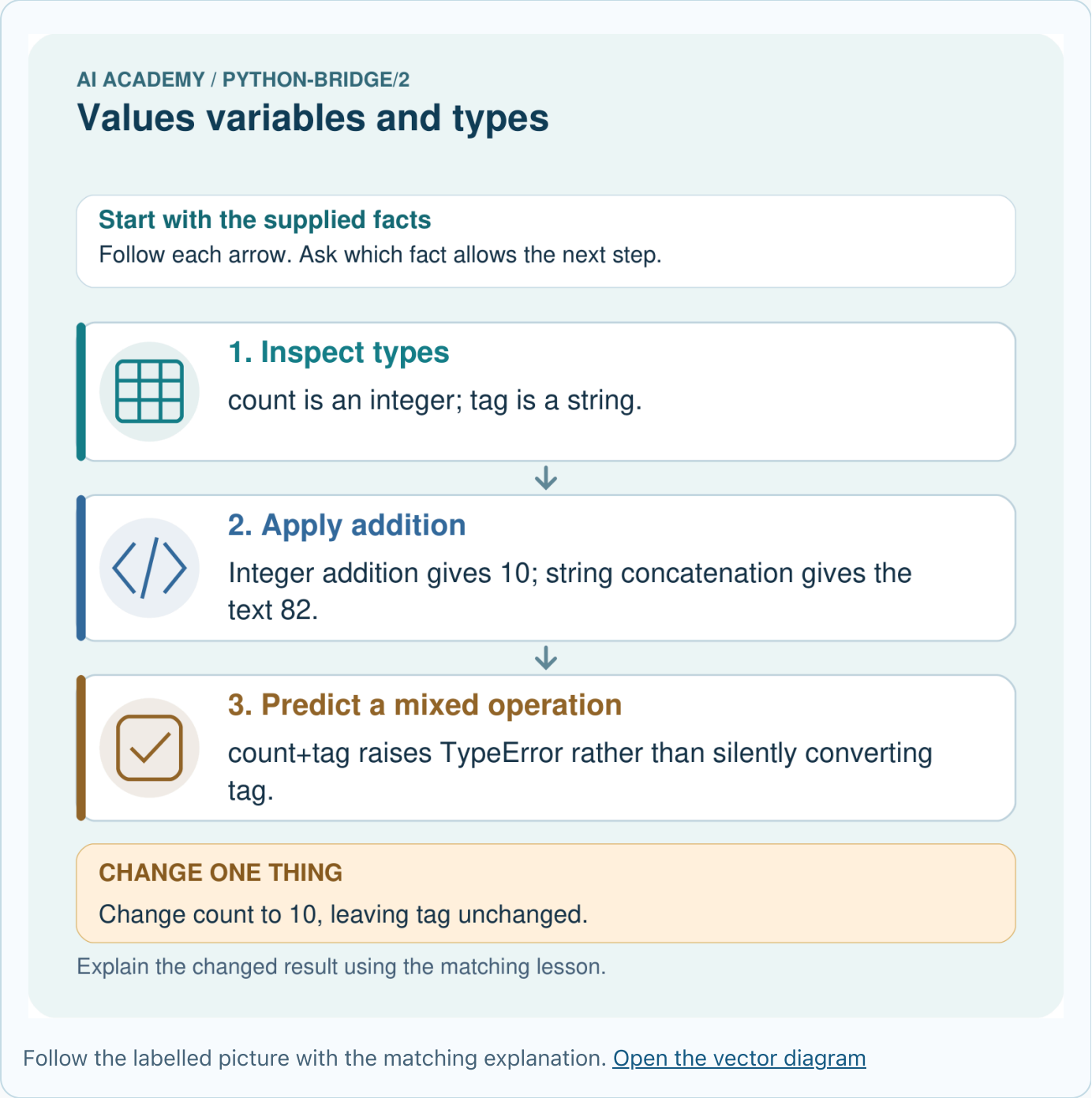
▼ **Earlier wording, alternatives and picture transcripts**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Values variables and types

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Distinguish a changed value from a changed type

> A valid type can still be wrong for the task.

Trace addition, joining and repetition

```
print(8 + 2)
print("8" + "2")
print("8" * 2)
```

These lines display 10, 82 and 88. Number addition calculates a sum. **String** addition joins text; **string** multiplication by a whole number repeats it. Python does not infer from the digits that you wanted a number.

Trying "8" + 2 instead raises **Type**Error because this addition mixes text and an integer.

Preserve identifiers and meaningful counts

For a supplied count, use a numeric literal such as 8. For a shelf ID, preserve text such as "08". Later weeks will parse and validate data read from files; changing a **type** alone will not prove that a **value** is valid.

Use descriptive names with underscores, such as shelf_id. Names are case-sensitive: count and Count are different names.

Predict the 6 and 3 expressions

- Predict 6 + 3, "6" + "3" and "6" * 3. Explain what happens for "6" + 3. Which expression counts six groups of three?

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise the saved-value trace with less help

> Record each name after each assignment, then compare actual output.

Explain the script in your own words

- Classify 12, 1.5, "12", False and "False". Explain why the last two are different.

After count becomes 10, add saved = count and print(saved). Predict the new output and explain why this is different from expecting saved to change automatically.

> **Explain your prediction to a partner before comparing results.**

Complete W2 prediction and trace

Use workbook W2 for the connected example "**Value**s **variable**s and **type**s". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Check digits-versus-number reasoning

- Use the worked case for **Value**s **variable**s and **type**s. Proposed explanation: "Digits always mean a number". Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured type change

- Predict both print **output**s and explain why they differ. Does count+tag work, and what happens to each valid **output** after count becomes ten?

My explanation using a pictured step and the given facts

Reserve independent work

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Trace a saved value

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Test five complete type and binding cases

> Choose an operation based on supplied types, then check whether it fits the task.

All supplied cases and expected boundaries

Complete supplied case	Prediction and meaning
A · count 8; saved=count; count=count+2; print both	10 then 8. saved captured the earlier integer.

Complete supplied case	Prediction and meaning
B · After A, assign saved=count, then print saved	10; the new assignment captures the current count.
C · Integers 6+3, strings "6"+"3", string "6"*3	9, text 63, text 666; count six groups of three with 6*3=18.
D · Evaluate "6"+3 in a separate fresh script	TypeError; neither silent conversion nor a displayed sum.
E · Count-meaning question for numeric -3 and text "08"	-3 is int but violates nonnegative-count meaning; "08" keeps identifier spelling.

Predict, run or trace and preserve errors

1. Run or trace the supplied type-inspection, saved-value and string-operation scripts separately. Predict A/B before editing and use the table for complete extra cases C/D/E. A type-inspection line prints type names, not the original data value; add print(shelf) only as a recorded proposed change if you want the visible 08. Keep D separate because its expected error stops later statements.

   Trace reassignments as new bindings, not permanent links. Preserve actual errors and note the task meaning separately from Python type. Keep the selected count/note/estimate/fits script unused until the individual attempt.

Original complete practice scripts

```
count = 8
shelf = "08"
print(type(count))
print(type(shelf))
```

```
count = 8
saved = count
count = count + 2
```

```
print(count)
print(saved)
```

```
print(8 + 2)
print("8" + "2")
print("8" * 2)
```

Keep the evidence record

Trace the five-line count and saved program. Record both printed **output**s, then predict the result of assigning saved = count after the change. Check with a run or label a paper trace.

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace the fresh selected values and reassignments

> Each statement uses the values bound when that line executes.

Selected complete task and scope

Predict both **output** lines and explain why estimate stays unchanged when count is reassigned. Repeat the trace with initial count=0. Explain why note is text even though its printed characters look like a number.

```
count = 4
note = "4"
estimate = count / 2
fits = count <= 5
count = count + 3
print(count, note, estimate, fits)
print(type(count).__name__, type(note).__name__, type(estimate).__name__,
type(fits).__name__)
```

Trace this exact selected script from the first line each time. When changing the initial count, leave the quoted note unchanged. Keep value/type predictions separate from an actual run and explain which earlier computations would need to run again to change.

Your response and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Additional practice · Earlier type task

A fresh script runs seats = 12; booked = seats; seats = seats - 5; print(seats); print(booked), with each statement on a separate line. Predict both **output**s. Then add booked = seats + 1 and print(booked).

Explain the evaluation order and choose a **type** for a seat label "07".

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Trace values, types and reassignment

Attempt this without hints

Use feedback to improve

Keep value, type and meaning on the same evidence sheet

> An identifier and a measurement have different task roles.

This week’s contribution

Add a labelled **value**s **variable**s and **type**s **evidence** sheet to Python readiness portfolio. Show how you can distinguish integer, floating-point, text and boolean **value**s.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week’s record; a matching answer without a trace is insufficient.

Project checkpoint and evidence

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

For Python readiness before the Responsible Plant Classifier, retain a trace of the **value**s, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

Finish: explain the value, its type and when it was computed

> Reassigning one name does not rerun earlier expressions.

Explain without notes

- Close your notes. Explain this goal in your own words: Distinguish integer, floating-point, text and Boolean **value**s

> **My deciding step, evidence and one remaining question**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

Reference · The words used in this procedure

> Explain each term using a particular code line.

Vocabulary and response

Word	Meaning
Value	A piece of data such as 8 or a text label
Variable	A name bound to a value during execution
Type	A category of value that determines available operations
String	A sequence of text characters
Reassignment	Binding an existing name to a new value

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain, apply and challenge

REFERENCE · SECTION 10

Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

Choose a challenge

- Classify 12, 1.5, "12", False and "False". Explain why the last two are different.
- Predict 6 + 3, "6" + "3" and "6" * 3. Explain what happens for "6" + 3. Which expression counts six groups of three?
- Give a suitable **type** for a counter count, a length of 2.5 metres, a shelf ID with a leading zero, and a yes-or-no availability flag. Give a count **value** with the right **type** that should still be rejected.

Explain which evidence supports your conclusion and which claim remains unproved.

▼ Earlier wording, alternatives and picture transcripts

Explain → apply → challenge

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

Self-check and explanation

- A negative int is still an integer, but it may be invalid as a count of objects. A correctly **type**d label can still name the wrong shelf. Keep the task's meaning separate from the **value**'s Python **type**.

  Week 3 uses arithmetic expressions and grouping to calculate with known numeric **value**s.

- Give a suitable **type** for a counter count, a length of 2.5 metres, a shelf ID with a leading zero, and a yes-or-no availability flag. Give a count **value** with the right **type** that should still be rejected.

> **A shelf identifier "08" is changed to the number 8 so that it can be added to another identifier. Explain what information is lost and why numeric addition does not solve an identifier task. Give one example of a value with the right type but the wrong meaning.**

My limited claim and a useful next check

▼ **Extra practice · A complete trace**

REFERENCE · SECTION 13

Extra practice · A complete trace

> Predict before opening the supplied trace.

Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Check the complete trace

A saved value is not a live link

INSTRUCTION	COUNT NOW	SAVED NOW
count = 8	8	Not assigned
saved = count	8	8
count = count + 2	10	8

Evaluate the right side using the current value, then bind the left-hand name. The saved integer does not update when count is reassigned.

▼ Earlier wording, alternatives and picture transcripts

Complete evidence record

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

Checklist and first prediction

By the end, I can	How I will show it
Distinguish integer, floating-point, text and Boolean values	Explain the deciding step, show a trace or test, and state one limit.
Trace reassignment without treating a name as a live formula	Explain the deciding step, show a trace or test, and state one limit.
Choose a suitable representation for a count and an identifier	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A number and its printed-looking text. A program stores a numeric badge count and a **string** that looks like that count.

▼ Extra practice · Predict, check and repair

REFERENCE · SECTION 17

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- A number and its printed-looking text. A program stores a numeric badge count and a **string** that looks like that count.
- Which labelled step matches this detail? count+tag raises **Type**Error rather than silently converting tag.
- 1. Predict a mixed operation
- 2. Inspect **type**s
- 3. Apply addition
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> _____
>
> _____
>
> _____
>
> _____

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Predict a mixed operation count+tag raises **Type**Error rather than silently converting tag. **Value**s carry **type**s, not merely appearances.

The plus operator performs numeric addition for these integers and concatenation for these **string**s; the mixed operation is invalid here.

Changed-case question and hint

- Changed-case practice: Change count to 10, leaving tag unchanged. Which conclusion is supported by the supplied facts?
- 1. This one example guarantees correct results for every future **input**.
- 2. We can decide the result without examining the changed **input** or the procedure.
- 3. The two valid **output**s become 12 and 82.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: The two valid **output**s become 12 and 82. Only the integer **variable** changes; the **string** still contains the character 8.

Repair and keep the record

Review From a paper algorithm to Python (python-bridge/1). Goal: Translate a short paper calculation into a Python script. Short practice: Match each paper quantity to its Python name.

Why does the spare count get added once, and what prints when spare is five?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ **Earlier wording, alternatives and picture transcripts**

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

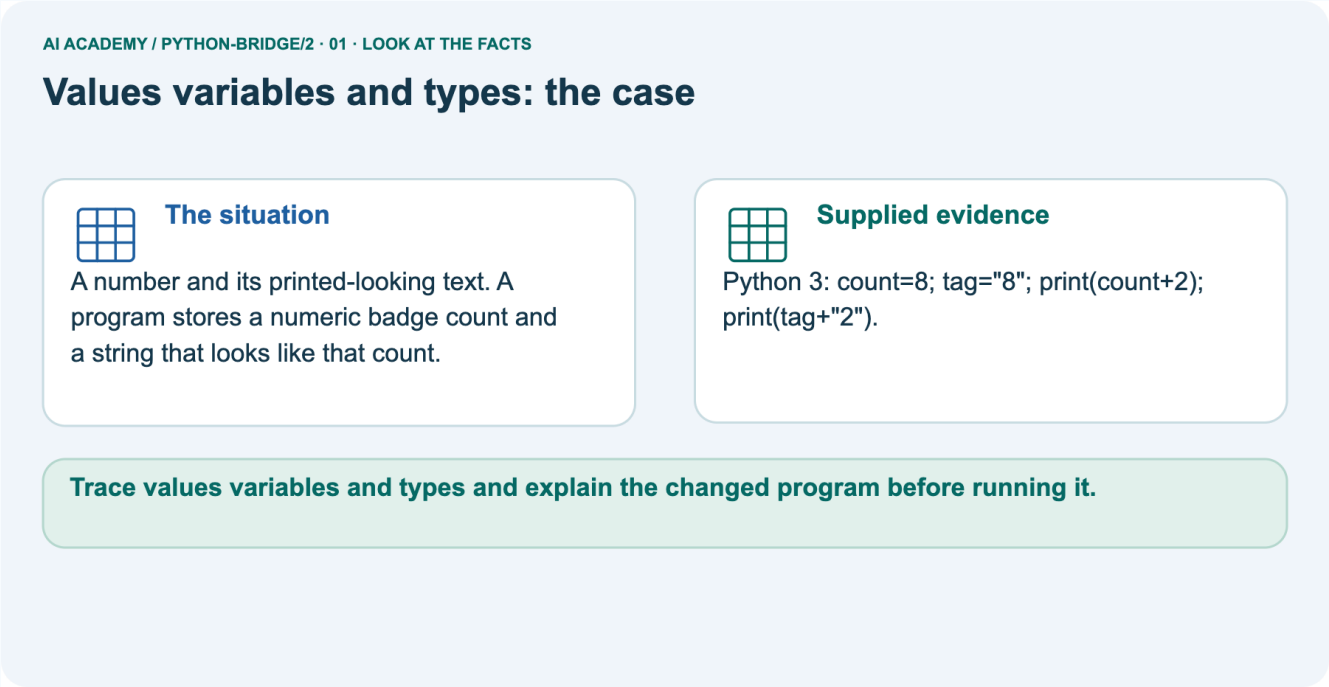
▼ **Reference · The six-picture journey**

REFERENCE · SECTION 18

Reference · The six-picture journey

> Choose the picture needed for the present question.

01. Values variables and types: the case

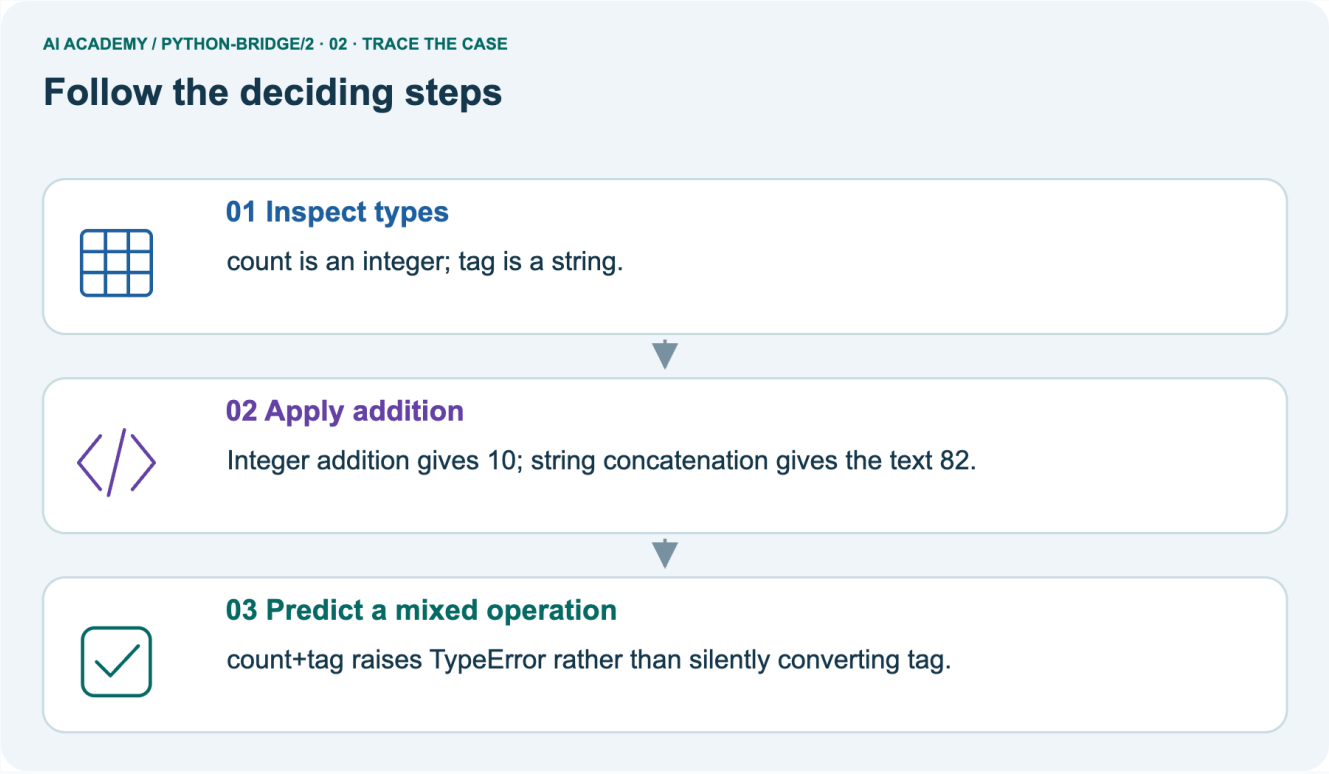


Follow the labelled picture with the matching explanation. Open the vector diagram

A number and its printed-looking text. A program stores a numeric badge count and a **string** that looks like that count. Supplied facts: Python 3: count=8; tag="8"; print(count+2); print(tag+"2").

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

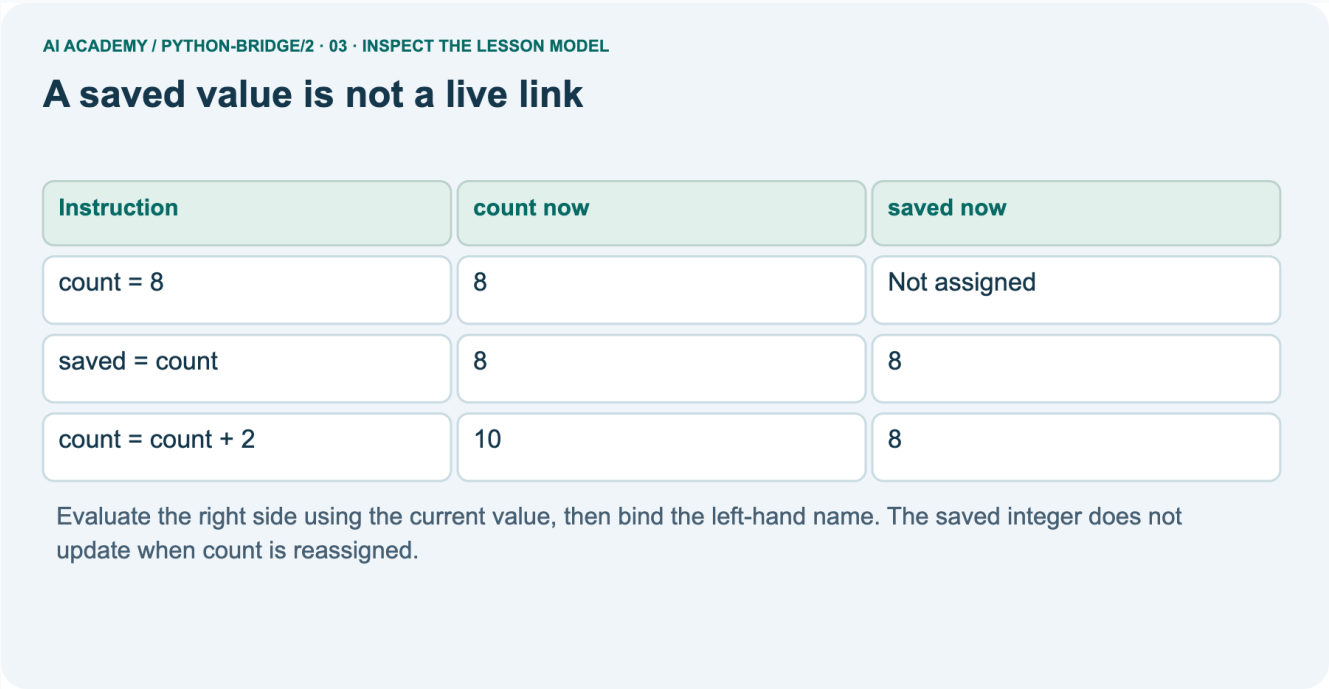


Follow the labelled picture with the matching explanation. Open the vector diagram

Inspect **type**s: count is an integer; tag is a **string**. → Apply addition: Integer addition gives 10; **string** concatenation gives the text 82. → Predict a mixed operation: count+tag raises **Type**Error rather than silently converting tag.

Point to each numbered stage and explain which recorded **input** it uses.

03. A saved value is not a live link

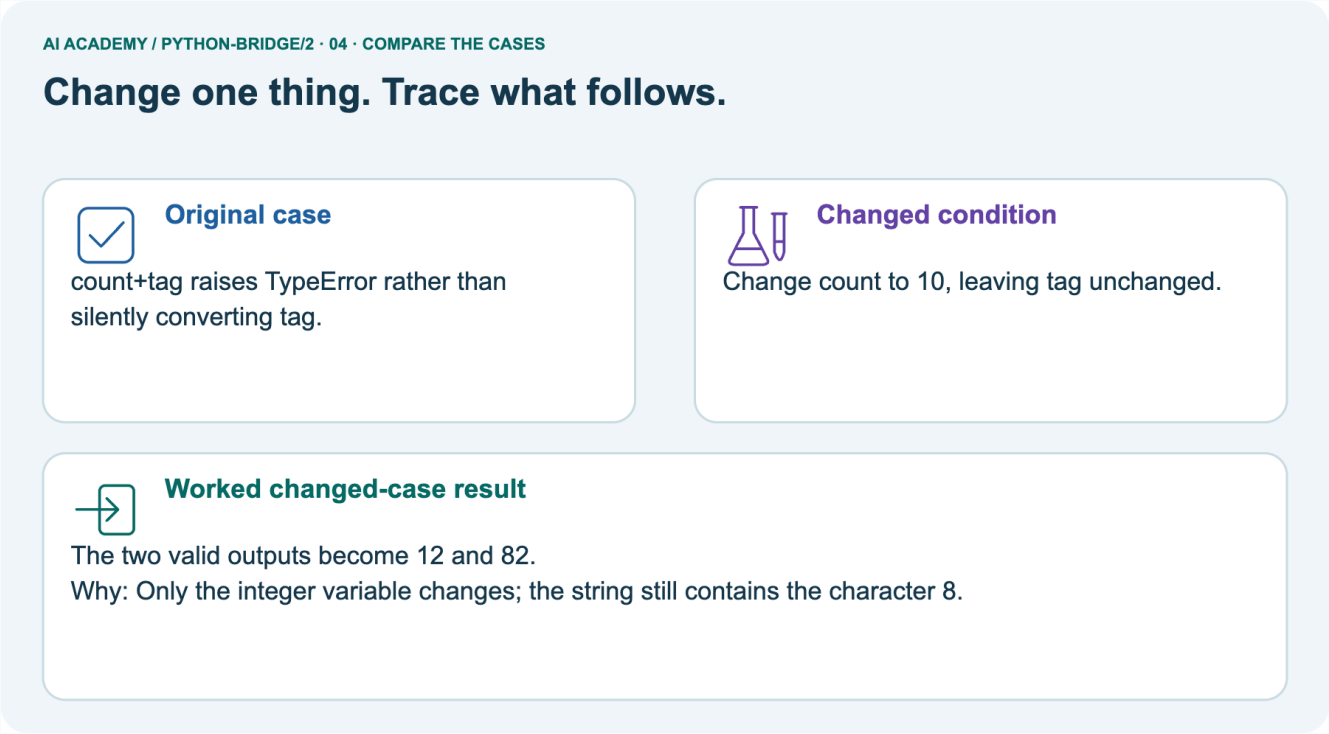


Follow the labelled picture with the matching explanation. Open the vector diagram

A saved **value** is not a live link. count = 8 | 8 | Not assigned. saved = count | 8 | 8. count = count + 2 | 10 | 8

Read the labels and **value**s. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



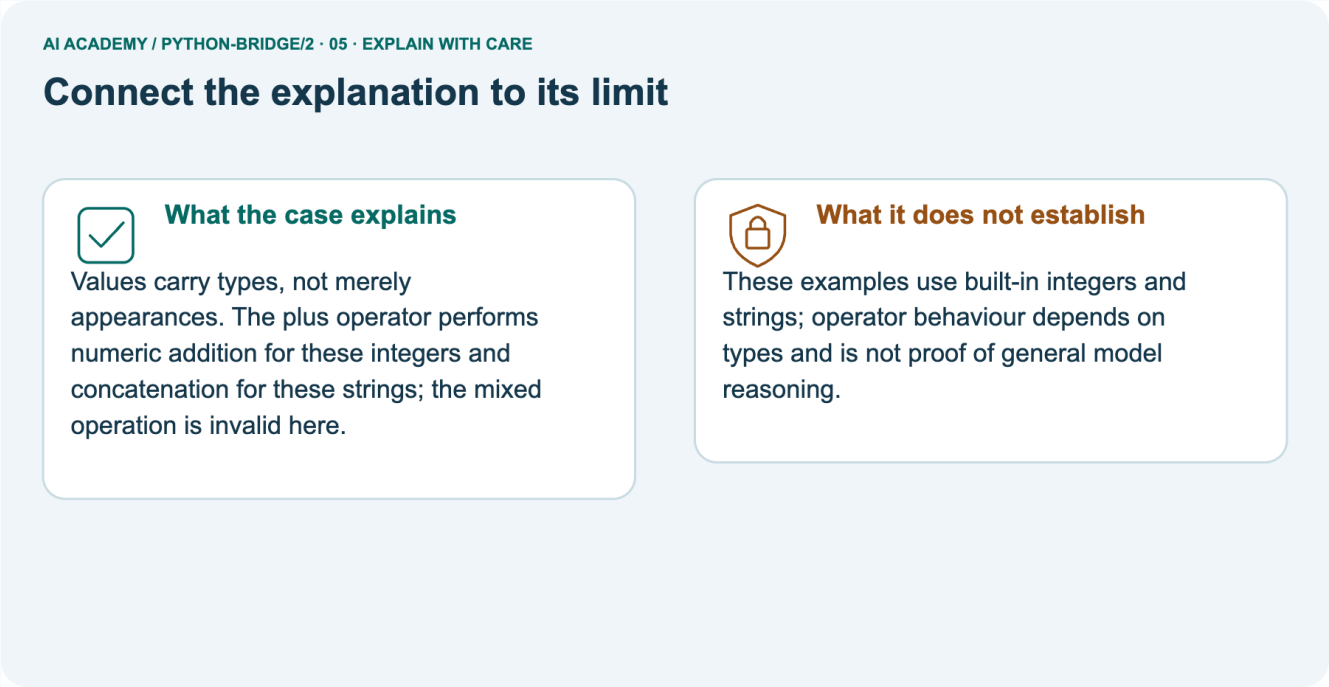
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: count+tag raises **Type**Error rather than silently converting tag. Changed condition: Change count to 10, leaving tag unchanged. Expected result: The two valid **output**s become 12 and 82.

Reason: Only the integer **variable** changes; the **string** still contains the character 8.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



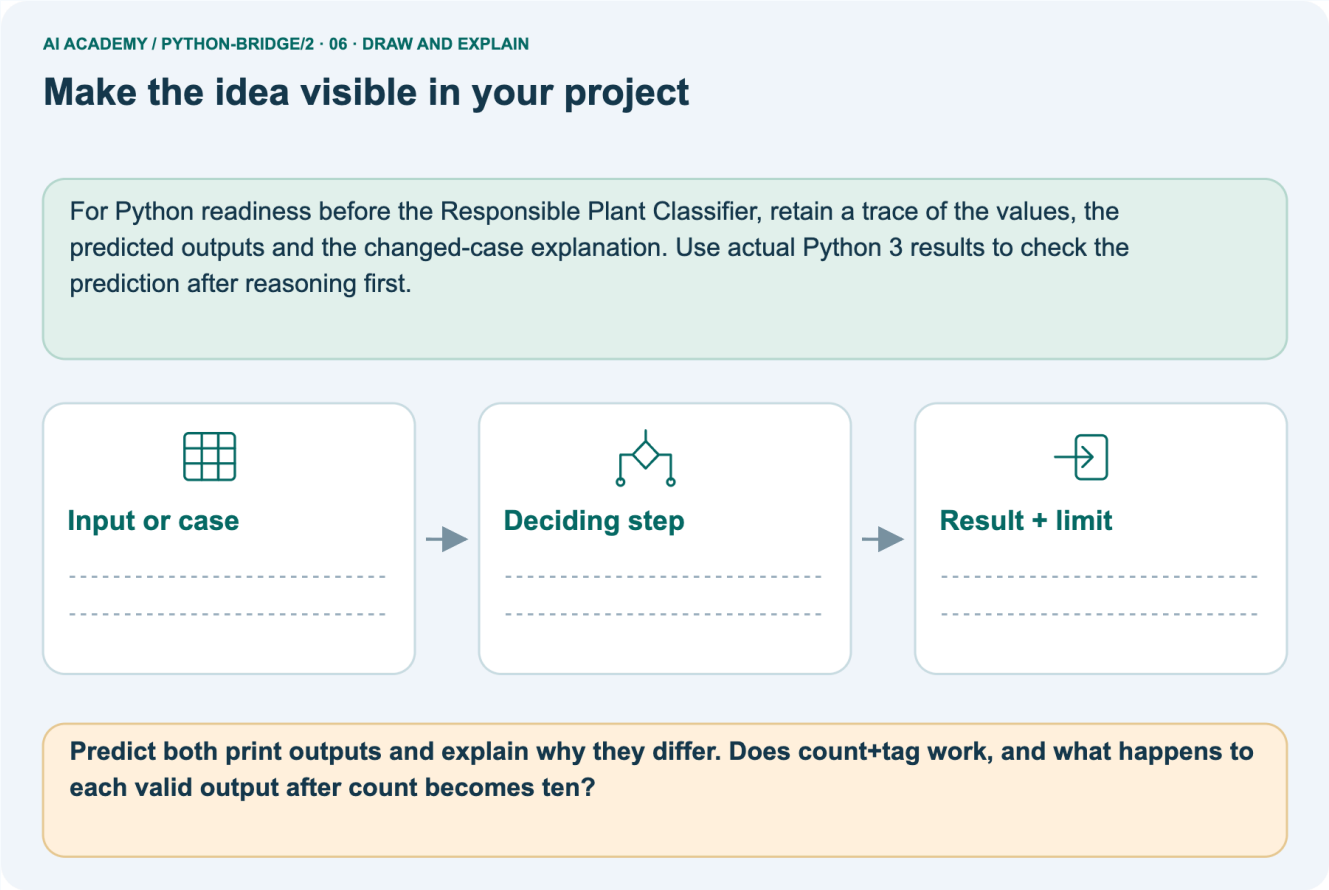
Follow the labelled picture with the matching explanation. Open the vector diagram

Values carry **type**s, not merely appearances. The plus operator performs numeric addition for these integers and concatenation for these **string**s; the mixed operation is invalid here.

Limit: These examples use built-in integers and **string**s; operator behaviour depends on **type**s and is not proof of general model reasoning.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the **value**s, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Predict both print **output**s and explain why they differ. Does count+tag work, and what happens to each valid **output** after count becomes ten?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Values variables and types: the case

02. Follow the deciding steps

03. A saved value is not a live link

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.